

Diagramming as code

Table of Contents

summary

History

- Early Developments

- The Rise of Text-Based Diagramming

- Integration with Development Practices

- Current Trends and Future Directions

Key Concepts

- Overview of Diagram as Code

- Application in Software Development

Tools and Technologies

- Diagram as Code Libraries

 - Mermaid

 - PlantUML

- Benefits of Diagram as Code

Use Cases

- Definition and Purpose

- Use Case Template

- Granularity of Use Cases

- Use Case Modeling Best Practices

Advantages

- Enhanced Version Control

- Reusability and Consistency

- Improved Communication

- Scalability and Collaboration

- Regular Review and Validation

- Clarity and Documentation Quality

- Dynamic Documentation

Challenges

- Use Case Granularity

- Diverse Learning Levels

Integration of Technologies
Emotional and Practical Barriers
Recap and Reinforcement

Best Practices

Tone and Voice
Consistency in Terminology
Structure and Organization
Clarity and Readability
Review Processes
Diagramming Tools
Meaningful Naming Conventions

Future Trends

Integration with CI/CD Pipelines
Enhanced Version Control
Increasing Popularity of Code-to-Diagram Tools
Emphasis on Collaborative Development
Educational Integration
Challenges and Considerations

Check <https://storm.genie.stanford.edu/article/688303> for more details

Stanford University Open Virtual Assistant Lab

The generated report can make mistakes.

Please consider checking important information.

The generated content does not represent the developer's viewpoint.

summary

Diagramming as Code is an innovative methodology that allows users to create diagrams through text-based code rather than traditional graphical tools. This approach enhances the integration and documentation processes in software development, offering significant advantages such as improved version control, collaboration, and clarity in representing complex systems. As software projects have grown in complexity, the limitations of conventional diagramming methods—such as difficulty in maintaining accurate and up-to-date visual representations—have prompted the adoption of this paradigm, which aligns closely with modern development practices like Agile and DevOps.^{[1][2][3]}

The concept gained momentum in the early 2010s with the rise of text-based diagramming tools and libraries, allowing developers to embed visual documentation directly within their codebases. Tools like Mermaid and PlantUML exemplify this trend by enabling the generation of diagrams from simple text descriptions, thus facili-

tating better collaboration among team members and ensuring that documentation evolves in tandem with code.[\[4\]\[5\]\[6\]](#). Diagramming as Code has become essential in fostering effective communication among technical and non-technical stakeholders, making it easier to convey system requirements and design structures.[\[7\]\[8\]](#).

Despite its advantages, Diagramming as Code is not without challenges. Issues such as determining the appropriate granularity of use cases, addressing diverse learning levels in educational settings, and integrating various technologies can complicate the effective use of this methodology. Additionally, the emotional and practical barriers that learners face when adopting new coding practices can hinder progress.[\[9-\]\[10\]\[11\]](#). Nonetheless, the methodology's focus on dynamic documentation and real-time updates positions it well for future trends, including deeper integration with Continuous Integration and Continuous Deployment (CI/CD) pipelines and greater collaboration in remote work environments.[\[12\]\[13\]](#).

As Diagramming as Code continues to evolve, its impact on software development and system architecture is expected to grow, emphasizing the importance of visual communication in an increasingly complex digital landscape. The integration of artificial intelligence and automated tools into the diagramming process suggests a future where the creation of visual representations becomes even more accessible and streamlined, thereby enhancing overall development workflows.[\[14\]\[15\]\[16\]](#).

History

The concept of Diagram as Code emerged as a response to the evolving needs of software development and system architecture in the 21st century. As projects grew in complexity, traditional diagramming methods faced challenges related to version control, collaboration, and the integration of diagrams into the development workflow. This led to the exploration of automated and code-based approaches to diagram creation.

Early Developments

Initial explorations into using code for diagramming can be traced back to the growing prominence of version control systems in software development. The need for maintaining accurate and up-to-date representations of complex systems paved the way for the development of tools that allowed diagrams to be generated programmatically. The idea was to separate the process of diagram creation from manual graphical interfaces, which often led to inefficiencies and errors[\[1\]\[2\]](#).

The Rise of Text-Based Diagramming

The adoption of text-based diagramming tools began to gain traction in the early 2010s. This period saw the emergence of various frameworks and libraries that facilitated the creation of diagrams through code. These tools leveraged familiar programming languages, enabling developers to integrate visual documentation directly into their codebases. The concept appealed to developers due to its ability

to enhance collaboration, improve accuracy, and maintain documentation as code evolved[3][2].

Integration with Development Practices

As Agile methodologies and DevOps practices became mainstream, the need for real-time collaboration and rapid iteration of diagrams further emphasized the value of Diagram as Code. By enabling teams to visualize workflows, architectures, and data flows programmatically, these tools facilitated a more agile approach to documentation. This allowed for better communication among team members and a clearer understanding of complex systems[4][5].

Current Trends and Future Directions

Today, the Diagram as Code paradigm continues to evolve, fueled by advancements in AI and automated tools that allow users to create diagrams from natural language descriptions. This development represents the infancy of a broader trend towards integrating AI into diagramming processes, indicating that future iterations of Diagram as Code may become even more accessible and user-friendly. The growing presence of these tools highlights the ongoing demand for effective visual communication in an increasingly complex digital landscape[1][6].

Key Concepts

Overview of Diagram as Code

Diagram as Code is an innovative approach that allows users to create diagrams through coding instead of traditional graphic design tools. This method entails writing code in a text file to define the structure, components, and connections of diagrams, which are then transformed into graphical representations. This approach enhances the integration and documentation processes in software projects, particularly useful for architectural and flow diagrams[7][8].

Application in Software Development

In the context of software engineering, Diagram as Code serves as a powerful tool for illustrating concepts, structures, and workflows. By converting diagrammatic ideas into code, development teams can maintain clarity and efficiency in their documentation practices. This method not only simplifies the visualization of complex systems but also enhances the overall development framework through effective communication of technical information[9][10].

The Diagram as Code approach is increasingly recognized as essential for modern software development, bridging the gap between technical design and effective visual communication.

Tools and Technologies

In the realm of diagramming as code, various tools and technologies have emerged to facilitate the creation and management of diagrams through code. These tools not only streamline the diagramming process but also enhance collaboration, version control, and maintainability within software development projects.

Diagram as Code Libraries

Mermaid

Mermaid is a JavaScript-based diagramming and charting tool that enables users to generate diagrams directly from textual descriptions. It supports various types of diagrams including flowcharts, sequence diagrams, and Gantt charts, making it a versatile option for developers who prefer coding their visual representations^[2]. Its integration with various platforms such as Markdown allows for seamless inclusion of diagrams within documentation.

PlantUML

PlantUML is another powerful tool for creating UML diagrams using a simple and intuitive scripting language. It enables the generation of diagrams from plain text, which is particularly beneficial for teams practicing version control. By keeping diagrams in code format, teams can easily track changes and collaborate more effectively^[3].

Benefits of Diagram as Code

The adoption of diagram as code not only promotes better organization and clarity but also enhances collaboration among team members. By using code to generate diagrams, developers can ensure that visualizations are automatically updated to reflect changes in the codebase, thereby maintaining accuracy throughout the project lifecycle^{[3][2]}. Furthermore, this approach facilitates comprehensive documentation that is easy to understand and accessible to both technical and non-technical stakeholders^[12].

Use Cases

Use cases are essential components in the design and development of systems, particularly in software architecture. They provide a structured way to describe how users interact with a system and help capture the requirements and functionalities needed to meet user needs.

Definition and Purpose

A use case outlines a specific scenario in which an actor (a user or another system) interacts with the system to achieve a particular goal. It represents high-level functionalities and serves as a bridge between user requirements and system design, effectively modeling services and tasks the system must perform.[\[9\]](#) By documenting these interactions, use cases facilitate communication among stakeholders, enabling better understanding and validation of system functionalities.

Use Case Template

A typical use case template includes several key elements:

Use Case Name: A descriptive title that reflects the functionality.

ID: A unique identifier for the use case.

Goal: The primary objective the actor aims to achieve.

Actors: Entities that interact with the system.

Preconditions: Conditions that must be true before the use case can execute.

Postconditions: Conditions that will be true after the use case has executed.

Main Flow: The standard sequence of steps for the use case.

Alternative Flows: Possible deviations from the main flow that could occur.

Exceptions: Any exceptions or errors that may arise during execution.[\[13\]](#)

This structured approach allows for a clear and organized presentation of use case information, making it easier to understand the system's requirements.

Granularity of Use Cases

The granularity of use cases can vary depending on the system and stakeholder needs. A well-defined use case description is essential, detailing all possible scenarios and variations, which aids in comprehensive system documentation.[\[13\]](#) It is crucial to ensure that use case descriptions are coherent, concise, and free of jargon, facilitating better comprehension by all stakeholders involved.

Use Case Modeling Best Practices

Effective use case modeling involves several best practices:

Stakeholder Involvement: Identify and engage key stakeholders in the use case development process to ensure their goals and needs are met.

Prioritization: Use a top-down approach to identify and prioritize the most critical use cases for implementation.

Consistent Naming Conventions: Employ a consistent and meaningful naming convention for use cases and actors to enhance clarity.

Complementary Use of Diagrams: Utilize both diagrams and textual descriptions to provide varying levels of detail for different audiences.

Validation: Regularly review and validate use cases with stakeholders to confirm alignment with system requirements and project objectives.[\[13\]](#)

By following these practices, teams can develop a robust set of use cases that accurately reflect the desired functionality and improve overall project outcomes.

Advantages

Enhanced Version Control

One of the most significant advantages of Diagram as Code is its seamless integration with version control systems like Git. This allows for tracking changes, facilitating collaboration among team members, and easily rolling back to previous versions when necessary, ensuring that diagrams remain accurate and up-to-date[\[3\]](#).

Reusability and Consistency

Diagram as Code enables the creation of reusable templates and components, ensuring consistency across various diagrams. This feature is particularly beneficial for complex systems, where maintaining a unified visual language is crucial. It simplifies the process of producing similar diagrams for different system components or new projects, promoting a coherent documentation style[\[3\]](#).

Improved Communication

Using Diagram as Code fosters enhanced communication among stakeholders, developers, and designers. It provides a common language for discussing system requirements, allowing for better understanding across technical and non-technical team members[\[14\]](#). This shared language is vital for clarifying system functionalities and interactions with external entities, thereby supporting effective collaboration[\[14\]](#).

Scalability and Collaboration

By storing diagram code alongside project code in the same repository, Diagram as Code encourages collaboration among team members. This practice fosters a shared understanding of the system architecture and invites contributions from developers, designers, and other stakeholders, ensuring that diagrams accurately represent the current state of the system[\[3\]\[15\]](#).

Regular Review and Validation

Establishing a routine for reviewing and validating diagrams is critical, as it prevents them from becoming outdated or inaccurate over time. Regular checks should involve team members familiar with the system to ensure that visual representations align with actual implementations. Promptly addressing discrepancies helps maintain the reliability of documentation[\[3\]\[15\]](#).

Clarity and Documentation Quality

Incorporating documentation into the development lifecycle enhances its relevance by mirroring the latest code iterations. A focus on clarity, descriptive naming, and standardized templates amplifies the effectiveness of documentation, making it easier for both technical and non-technical individuals to understand[\[16\]\[17\]](#).

Dynamic Documentation

Diagram as Code encourages the use of automated tools for generating documentation, allowing for real-time updates that reflect the latest code changes. This results in a more dynamic experience for end-users and developers who rely on accurate and current documentation to comprehend software design and functionality[\[18\]](#).

These advantages underscore the importance of adopting Diagram as Code practices within software development, as they contribute to improved documentation accuracy, enhanced team collaboration, and ultimately, better software quality.

Challenges

Use Case Granularity

One significant challenge in diagramming as code involves the appropriate granularity of use cases. Project planning can be greatly affected by the level of detail in use cases, where smaller, more finely grained use cases facilitate more accurate time and effort estimations for development tasks, thus improving project scheduling and resource allocation[\[13\]](#). However, achieving the right balance is critical; if use cases are overly high-level, they may lack the detail needed for effective development, while excessively detailed use cases can become unwieldy and difficult to manage[\[13\]](#). Striking the right level of granularity is essential to ensure that use cases remain understandable and effective for all stakeholders involved.

Diverse Learning Levels

In educational settings, particularly in teaching coding or system design, instructors face the challenge of addressing diverse student learning levels. Given that students come from various backgrounds and possess different levels of understanding, it is crucial to focus lessons on the middle range of ability within a class[\[19\]](#). This approach helps to ensure that lessons are accessible to most students, but it also necessitates individualized attention for those who may struggle or excel. Incorporating tiered activities and providing additional resources can help accommodate varying learning paces and styles, promoting a more inclusive learning environment.

Integration of Technologies

Another challenge arises from the integration of multiple technologies and methodologies in diagramming systems. For instance, the creation of use case diagrams

(UCD) and class diagrams (CD) involves complex processes such as identifying actors, relationships, and classes, which can be overwhelming for students and practitioners alike[20]. Additionally, educators must adapt their teaching strategies to incorporate innovative elements like gamified learning and flipped classrooms, which may require significant adjustments in pedagogical approaches to meet students' varying needs[21][22]. Ensuring that all students can effectively engage with and comprehend these integrated systems is a persistent challenge in the field.

Emotional and Practical Barriers

Furthermore, emotional and practical barriers can impede learning in coding and diagramming practices. Factors such as anxiety, prior negative experiences, and external life circumstances can heavily influence student engagement and success[19]. Instructors are tasked with creating a supportive environment that acknowledges these challenges, implementing strategies to alleviate stress, and fostering resilience among students. For example, celebrating small achievements and providing opportunities for resubmission can help cultivate a growth mindset and encourage ongoing participation and improvement.

Recap and Reinforcement

Finally, a recurrent issue in educational settings is the need for effective recap and reinforcement of learning material. With a plethora of concepts to grasp in coding and diagramming, regular checks for understanding and summary sessions at the end of lessons are vital to solidify student comprehension[19]. Instructors must devise strategies to regularly engage students, ensuring they are following along and grasping key concepts throughout the learning process.

Best Practices

Tone and Voice

When creating diagrams as code, it's essential to maintain a formal tone and clear voice in your documentation. The use of active voice is encouraged for instructions to enhance clarity, such as stating "Run the command" instead of "The command should be run"[23]. This practice not only aids comprehension but also establishes a professional standard.

Consistency in Terminology

To avoid confusion, standardize terminology throughout your diagrams and documentation. This includes maintaining a glossary for terms, abbreviations, and acronyms specific to your project. Consistency in language, such as using "feature" rather than alternating with "functionality," helps to streamline communication[23].

Structure and Organization

Organizing your diagrams effectively is key to their usability. Utilize standardized templates and formats to ensure that your documentation is coherent and easy to follow[16]. Diagrams should be designed to mirror the development lifecycle, thereby maintaining relevance as the project evolves[16].

Clarity and Readability

Utilize clear, jargon-free language in your documentation to cater to both technical and non-technical audiences. The readability of your diagrams is paramount, as technical subjects can be complex[24]. To enhance clarity, limit the use of screenshots and ensure that they are relevant, as outdated visuals can erode user trust[25].

Review Processes

Implement structured review mechanisms to evaluate the quality of your diagrams and documentation. Regular peer feedback is crucial for identifying inaccuracies and understanding audience needs, which in turn enhances the quality of the output[26].

Diagramming Tools

Choose the right diagramming tools based on the context and requirements of your project. While traditional tools like Visio are suitable for initial sketches, code-driven tools are more effective for maintaining up-to-date diagrams in sync with the development process[5][24]. Diagrams-as-Code approaches allow for dynamic updates and better integration with the codebase, thereby improving collaboration and efficiency[7].

Meaningful Naming Conventions

Adopt descriptive naming conventions for elements within your diagrams. Meaningful names clarify the purpose and functionality of various components, which enhances understanding and usability for the audience[16].

By adhering to these best practices, you can ensure that your diagramming as code approach is effective, professional, and aligned with the needs of your audience.

Future Trends

Integration with CI/CD Pipelines

The future of diagramming as code is poised for significant advancements, particularly with its integration into Continuous Integration and Continuous Deployment (CI/CD) pipelines. This integration allows for automated diagram generation that synchronizes with code changes, enhancing the reliability and accuracy of visual documentation. As developers make modifications to their code, corresponding diagrams can automatically update, ensuring that all team members have access to the latest architectural insights without manual intervention[3][27].

Enhanced Version Control

Another emerging trend is the improved version control capabilities associated with diagram-as-code tools. By treating diagrams as code, developers can version control diagrams alongside their application code, allowing them to track changes and revert to previous versions as needed. This capability not only aids in maintaining a historical record of the project's evolution but also facilitates audits and architectural reviews, making it easier to manage complex systems over time[\[27\]\[28\]](#).

Increasing Popularity of Code-to-Diagram Tools

The growing demand for code-to-diagram tools is expected to continue, driven by the need for efficiency and accuracy in visual communication within development teams. These tools eliminate the cumbersome manual tweaking often associated with traditional diagramming methods, providing a streamlined approach that saves time and enhances clarity. As more developers recognize the benefits of generating diagrams directly from structured code, adoption rates are likely to rise[\[29\]\[30\]\[22\]](#).

Emphasis on Collaborative Development

With the shift towards remote and collaborative work environments, the need for tools that facilitate shared understanding among team members will be crucial. Diagramming as code supports this collaborative trend by enabling multiple contributors to work on visual representations of systems in real-time, enhancing teamwork and project alignment. This collaborative capability is particularly valuable in agile development settings, where continuous feedback and iteration are essential for success[\[28\]\[2\]](#).

Educational Integration

The future also points towards increased integration of diagramming as code in educational contexts. As coding becomes more prevalent in educational curricula, teaching students how to effectively use diagramming tools alongside coding practices will enhance their understanding of system architecture and design. This integration can foster a deeper appreciation for the relationships between code and its visual representations, ultimately preparing students for modern software development challenges[\[22\]\[27\]](#).

Challenges and Considerations

Despite the promising trends, challenges remain in the widespread adoption of diagramming as code. These include the learning curve associated with new tools, the need for robust documentation, and potential limitations in tooling options. However, the advantages offered by programmatically generated diagrams—such as improved clarity, consistency, and efficiency—suggest that overcoming these challenges is worthwhile for development teams seeking to enhance their workflows[\[30\]\[2\]](#).

References

- [1]: [Understanding Diagrams as Code | Gliffy by Perforce](#)
- [2]: [Diagram as Code - by Reza - DevSecOps Guides](#)
- [3]: [Diagram as Code: Visualizing Systems with Efficiency and Clarity - LinkedIn](#)
- [4]: [5 Reasons You Should Use Diagram as Code Solutions - ZenUML](#)
- [5]: [Plain text diagrams – the best diagrams as code tools - AugmentedMind.de](#)
- [6]: [Mastering the Best Practices of Technical Documentation - Floik](#)
- [7]: [Diagram-as-Code: Creating Dynamic and Interactive Documentation for ...](#)
- [8]: [Diagram-as-Code: Creating Dynamic and Interactive ... - GitHub](#)
- [9]: [UML Use Case Diagram: Tutorial with EXAMPLE - Guru99](#)
- [10]: [Efficient Code Documentation: Best Practices and Examples](#)
- [11]: [5 Top Free Diagramming Tools for Software Development](#)
- [12]: [Creating Effective Technical Documentation: Tips and Best Practices](#)
- [13]: [A Comprehensive Guide to Use Case Modeling - Visual Paradigm Guides](#)
- [14]: [UML Use Case Diagram: A Beginner's Guide - The Knowledge Academy](#)
- [15]: [Diagrams as Code: Intro for Developers - daily.dev](#)
- [16]: [Code Documentation Guidelines: Best Practices and Templates](#)
- [17]: [Limitations of UML? - Software Engineering Stack Exchange](#)
- [18]: [Mastering Coding Documentation: Best Practices and Examples](#)
- [19]: [How to Teach Code - We Learn Code](#)
- [20]: [Event-based approach for analyzing and designing system: A case study of designing curriculum system](#)
- [21]: [Overcome Challenges in Teaching Coding | 21K School](#)
- [22]: [How Can I Teach Coding Effectively: Strategies for Success](#)
- [23]: [Software Documentation Best Practices, Templates, Formats & Examples](#)
- [24]: [Diagramming Tools: When Should You Use No Code, DSL, or Code?](#)
- [25]: [Breaking down common documentation mistakes - mintlify.com](#)
- [26]: [Common Technical Writing Mistakes and How to Fix Them](#)
- [27]: [Top Diagram-as-Code Tools for Developers You Must Know About](#)
- [28]: [Creating Effective Flowcharts for Coding: A Step-by-Step Guide](#)
- [29]: [OLCreate: General Teaching Methods: Diagrams | OLCreate - OpenLearn](#)
- [30]: [Creately Plugin for Confluence | Creately](#)